Sentiment Analysis in Social Media

A case study on topic-sensitive classification of posts from X (Formerly Twitter)

Hernández Pérez Erick Fernando

Figure 1

Sentiment analysis, also known as opinion mining, is a subfield of natural language processing (NLP) that focuses on identifying and categorizing the emotional tone or subjective information contained within textual data. Its primary goal is to determine whether a piece of text expresses a positive, negative, or neutral sentiment. In more refined approaches, sentiments may be categorized along a broader spectrum or assigned to more specific emotions.

Understanding sentiment is essential for interpreting the intentions and perceptions of users, particularly in contexts where large volumes of text are generated, such as on social media platforms, in product reviews, or in customer feedback. Accurate sentiment analysis can provide valuable insights into public opinion, consumer satisfaction, and social trends.

Potential Applications

The results of sentiment analysis can be applied in a variety of domains, including:

- **Brand Monitoring**: Companies can track public opinion and quickly respond to negative sentiment about their products or services.
- **Political Analysis**: Analysts can gauge public reaction to political events, policies, or speeches.
- **Market Research**: Businesses can analyze consumer trends and identify emerging demands or pain points.
- **Public Health**: Health organizations can monitor sentiment around health crises, misinformation, or public compliance.
- **Crisis Management**: Institutions can identify spikes in negative sentiment that signal unrest or dissatisfaction and act accordingly.

Overall, sentiment analysis serves as a powerful tool for understanding human expression at scale, providing actionable insights across disciplines from business to social sciences.

Case Study: Sentiment Analysis on Posts from X (formerly Twitter)

In this study, we examine a set of posts extracted from the social media platform X (formerly known as Twitter). The goal is to classify each post according to its sentiment in the context of a specific topic or hashtag. The possible sentiment labels are:

- **Positive**: expressing support, approval, or enthusiasm regarding the topic.
- **Negative**: expressing discontent, criticism, or rejection.
- **Neutral**: factual or emotionless posts with no clear sentiment.
- **Irrelevant**: unrelated to the selected topic or containing off-topic content.

To perform this classification, a preprocessing pipeline was implemented to clean the text data, remove noise (such as links and mentions), and normalize the language. Sentiment classification was carried out using a supervised learning model trained on labeled datasets, and performance was evaluated using standard metrics such as accuracy, precision, recall, and F1-score.

Initial Preprocessing and Data Preparation

The first and one of the most crucial steps in any sentiment analysis pipeline is preprocessing. Raw text data collected from social media platforms—such as posts on X (formerly Twitter)—typically contains a wide variety of noise: HTML tags, mentions, URLs, emojis, special characters, and inconsistent casing. These elements can mislead machine learning models and reduce the quality of the extracted features.

In this project, we begin by importing the necessary libraries and tools for data handling, text normalization, and model building. The dataset used is retrieved directly from the Kaggle repository `jp797498e/twitter-entity-sentimen` using the `kagglehub` package.

To clean the data, we define a custom preprocessing function that performs the following tasks:

- Removes HTML tags and URLs.
- Removes mentions (usernames) and hashtag symbols.
- Eliminates non-alphabetic characters and converts the text to lowercase.
- Extracts and preserves common emoticons, which can be strong indicators of sentiment.
- Applies stemming using the `SnowballStemmer` to reduce words to their root forms.

Below is a simplified version of the preprocessing pipeline:

```
def preprocessor(text):
    text = re.sub('<[^>]*>', '', text)
    emoticons = re.findall(r'(?::|;|=)(?:-)?(?:\ )|\ (|D|P)', text)
    text = re.sub(r'http\S+|www.\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#', '', text)
    text = re.sub(r'[\W]+', ' ', text.lower())
    words = text.split()
    stemmed_words = [stemmer.stem(word) for word in words]
    text = ' '.join(stemmed_words)
    text += ' ' + ' '.join(emoticons).replace('-', '')
    return text
```

Once the function is defined, we load and preprocess both the training and validation datasets. Sentiment labels are mapped to integers as follows:

`0`	Negative
`1`	Neutral
`2`	Positive
`3`	Irrelevant

We also define a helper function `prepare_data()` that optionally removes irrelevant or neutral posts depending on the desired scope of classification. This allows for flexibility in focusing the analysis on polarized opinions (positive vs. negative) when required.

```
def prepare_data(df, remove_irrelevant=False, remove_neutral=False):
    if remove_irrelevant:
        df = df[df['Sentiment'] != 3]
    if remove_neutral:
        df = df[df['Sentiment'] != 1]
    X = df['Text'].apply(preprocessor)
    y = df['Sentiment']
    return X, y
```

This preprocessing phase sets a strong foundation for building robust and reliable sentiment classification models by standardizing and simplifying the text input.

```
import kagglehub
import pandas as pd
import re

from nltk.stem import SnowballStemmer
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report

# Initialize stemmer
stemmer = SnowballStemmer('english')

# Custom text preprocessing
def preprocessor(text):
    text = re.sub('<[^>]*>', '', text)
    emoticons = re.findall(r'(?::|;|=)(?:-)?(?:\ )|\ (|D|P)', text)
    text = re.sub(r'http\S+|www.\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#', '', text)
    text = re.sub(r'[\W]+', ' ', text.lower())
    words = text.split()
    stemmed_words = [stemmer.stem(word) for word in words]
    text = ' '.join(stemmed_words)
    text += ' ' + ' '.join(emoticons).replace('-', '')
    return text

# Download dataset
path = kagglehub.dataset_download("jp797498e/twitter-entity-sentiment-analysis")

df_train = pd.read_csv(path + '/twitter_training.csv',
↪   header=None).drop(columns=[0]).rename(columns={1: "Topic", 2: "Sentiment", 3:
↪   "Text"})
df_validation = pd.read_csv(path + '/twitter_validation.csv',
↪   header=None).drop(columns=[0]).rename(columns={1: "Topic", 2: "Sentiment", 3:
↪   "Text"})

# Label mapping
sentiment_map = {'Negative': 0, 'Neutral': 1, 'Positive': 2, 'Irrelevant': 3}
df_train['Sentiment'] = df_train['Sentiment'].map(sentiment_map)
df_validation['Sentiment'] = df_validation['Sentiment'].map(sentiment_map)
```

```
# Clean text columns
df_train['Text'] = df_train['Text'].fillna('').astype(str)
df_validation['Text'] = df_validation['Text'].fillna('').astype(str)

# Data preparation function
def prepare_data(df, remove_irrelevant=False, remove_neutral=False):
    if remove_irrelevant:
        df = df[df['Sentiment'] != 3]
    if remove_neutral:
        df = df[df['Sentiment'] != 1]
    X = df['Text'].apply(preprocessor)
    y = df['Sentiment']
    return X, y
```

Exploratory Analysis of Discussion Topics

Before training any classification model, it is essential to understand the distribution of the data. One important aspect of this exploration is identifying the most frequently discussed topics in the dataset. Each post in the dataset is associated with a specific `Topic` label, which typically refers to a named entity or general subject such as a company, public figure, or event.

To visualize the prevalence of different topics, we use horizontal bar plots that show the number of occurrences of each topic in both the training and validation sets.

The following Python code generates the topic frequency plots using `matplotlib` and `seaborn`:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

# -------------------
# TRAIN set plot
# -------------------
plt.figure(figsize=(10, 6))
train_topic_counts = df_train['Topic'].value_counts().sort_values(ascending=False)
sns.barplot(x=train_topic_counts.values, y=train_topic_counts.index, palette='viridis')
plt.title("Topic Frequency in Training Set")
plt.xlabel("Number of Examples")
plt.ylabel("Topic")
plt.show()

# -------------------
# VALIDATION set plot
# -------------------
plt.figure(figsize=(10, 6))
val_topic_counts = df_validation['Topic'].value_counts().sort_values(ascending=False)
sns.barplot(x=val_topic_counts.values, y=val_topic_counts.index, palette='magma')
plt.title("Topic Frequency in Validation Set")
plt.xlabel("Number of Examples")
plt.ylabel("Topic")
plt.show()
```

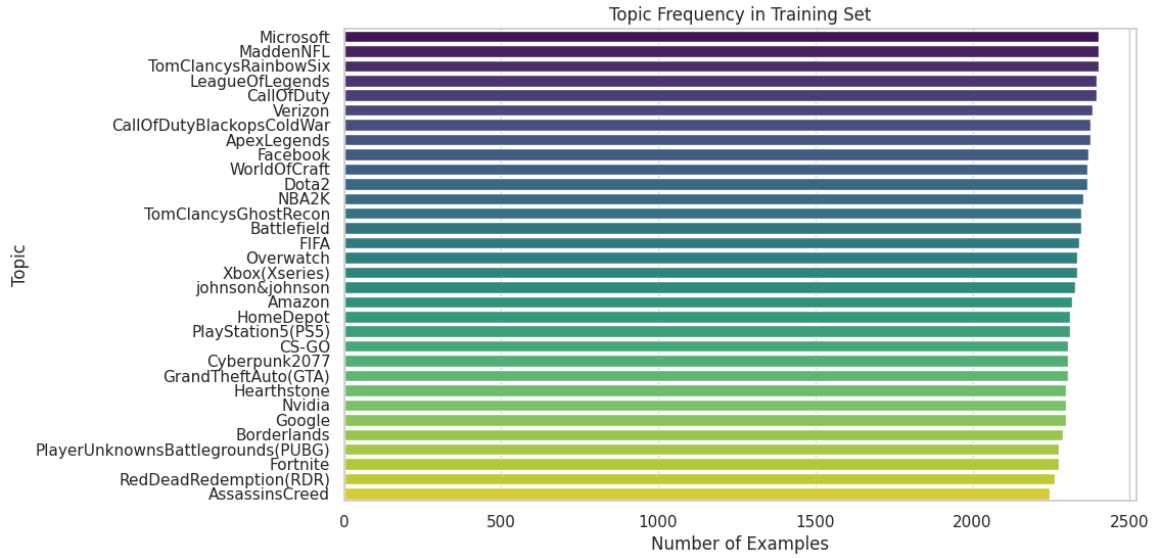

Figure 2

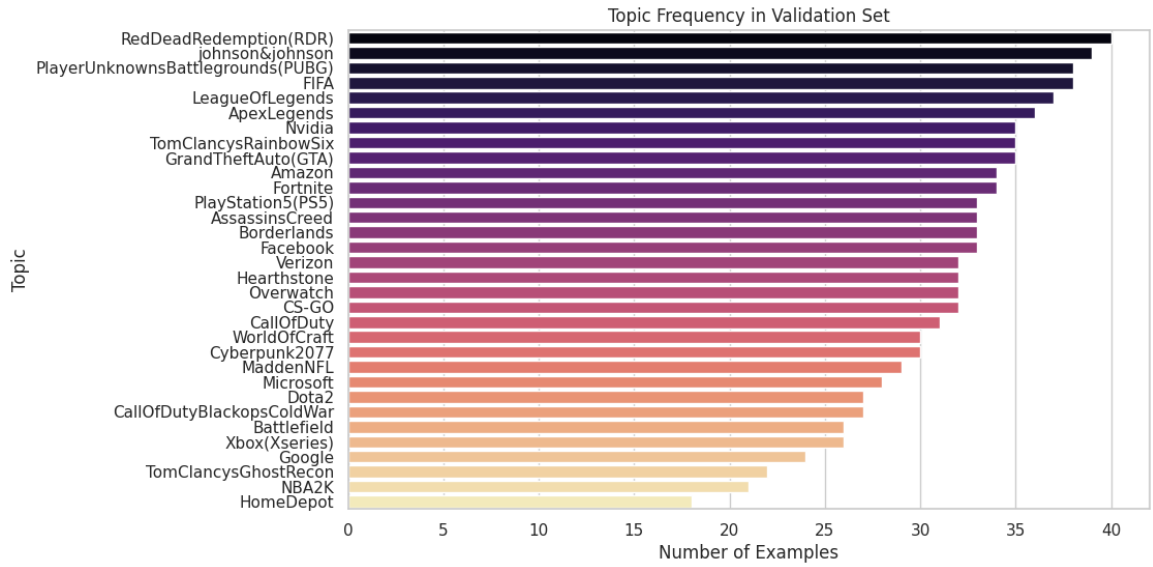

Figure 3

Modeling and Experimental Design

To classify the sentiment of social media posts, we implement and evaluate two widely used linear classification algorithms: **Support Vector Machines (SVM)** and **Logistic Regression**. Both models are well-suited for high-dimensional sparse data such as text, particularly when combined with a `TF-IDF` vectorizer.

Classification scenarios

In order to thoroughly understand the behavior and robustness of our models, we design three classification scenarios of increasing complexity:

1. **Binary Classification (Positive vs. Negative):** In this setting, we focus only on posts that express clearly polarized sentiments. Neutral and irrelevant posts are excluded. This scenario provides a baseline for evaluating the ability of models to distinguish between opposite sentiments.

2. **Ternary Classification (Positive, Negative, Neutral):** Here, we reintroduce neutral posts. This makes the task more realistic, as many social media posts are not explicitly emotional but still carry informative content. The inclusion of a neutral class introduces ambiguity and subtlety, challenging the decision boundaries of the classifiers.
3. **Full Classification (Positive, Negative, Neutral, Irrelevant):** This is the most comprehensive scenario. In addition to opinionated and neutral posts, it includes irrelevant messages—posts that do not pertain to the topic at hand or provide no clear sentiment. This reflects a real-world application where incoming data cannot always be filtered beforehand.

These three settings serve both practical and academic purposes. In many real applications, sentiment classifiers must operate under different constraints. For example:

- A brand monitoring system might only care about *positive vs. negative* opinions.
- A public opinion analysis may benefit from distinguishing neutral responses from polarized ones.
- A fully automated pipeline must be able to detect and discard irrelevant content, making a four-class classification necessary.

By evaluating both SVM and Logistic Regression across these scenarios, we aim to:

- Compare model performance under binary vs. multiclass setups.
- Assess the scalability and sensitivity of each model to increased complexity.
- Understand the trade-offs between precision and generality.

This staged approach provides a structured path toward building a sentiment analysis system adaptable to various real-world constraints.

Implementation and Model Selection

```
# ============================
# Datasets for each scenario
# ============================
# 1. 4-class (includes neutral and irrelevant)
X_train_all, y_train_all = prepare_data(df_train, remove_irrelevant=False,
↪  remove_neutral=False)
X_val_all, y_val_all = prepare_data(df_validation, remove_irrelevant=False,
↪  remove_neutral=False)

# 2. 3-class (removes irrelevant)
X_train_neu, y_train_neu = prepare_data(df_train, remove_irrelevant=True,
↪  remove_neutral=False)
X_val_neu, y_val_neu = prepare_data(df_validation, remove_irrelevant=True,
↪  remove_neutral=False)

# 3. 2-class (removes neutral and irrelevant)
X_train_bin, y_train_bin = prepare_data(df_train, remove_irrelevant=True,
↪  remove_neutral=True)
X_val_bin, y_val_bin = prepare_data(df_validation, remove_irrelevant=True,
↪  remove_neutral=True)

# ============================
# Pipeline setup
# ============================
def get_pipeline(model):
    return Pipeline([
        ('tfidf', TfidfVectorizer(stop_words='english')),
        ('clf', model)
    ])
```

```
# Grid parameters
svm_grid = {
    'tfidf__ngram_range': [(1,1), (1,2)],
    'tfidf__max_df': [0.9, 1.0],
    'tfidf__min_df': [1, 3],
    'clf__C': [0.1, 1, 10]
}

logreg_grid = {
    'tfidf__ngram_range': [(1,1), (1,2)],
    'tfidf__max_df': [0.9, 1.0],
    'tfidf__min_df': [1, 3],
    'clf__C': [0.1, 1, 10],
    'clf__solver': ['liblinear']
}

# ============================
# Grid search execution
# ============================
def run_grid_search(X_train, y_train, X_val, y_val, model, grid, label):
    print(f"\nGrid Search {label}...")
    pipe = get_pipeline(model)
    gs = GridSearchCV(pipe, grid, cv=3, scoring='accuracy', verbose=2, n_jobs=-1)
    gs.fit(X_train, y_train)
    print("Best parameters:", gs.best_params_)
    print("Best cross-val score:", gs.best_score_)

    y_pred = gs.predict(X_val)
    print(f"\nVALIDATION ({label}):")
    print(classification_report(y_val, y_pred))

# ============================
# Run for all 3 scenarios and 2 models
# ============================

# ---- SVM ----
run_grid_search(X_train_all, y_train_all, X_val_all, y_val_all, LinearSVC(), svm_grid,
↪   "SVM - 4 classes (with neutrals and irrelevants)")
run_grid_search(X_train_neu, y_train_neu, X_val_neu, y_val_neu, LinearSVC(), svm_grid,
↪   "SVM - 3 classes (no irrelevant)")
run_grid_search(X_train_bin, y_train_bin, X_val_bin, y_val_bin, LinearSVC(), svm_grid,
↪   "SVM - 2 classes (binary without neutral or irrelevant)")

# ---- Logistic Regression ----
run_grid_search(X_train_all, y_train_all, X_val_all, y_val_all,
↪   LogisticRegression(max_iter=1000), logreg_grid, "LogReg - 4 classes (with neutrals
↪   and irrelevants)")
run_grid_search(X_train_neu, y_train_neu, X_val_neu, y_val_neu,
↪   LogisticRegression(max_iter=1000), logreg_grid, "LogReg - 3 classes (no irrelevant)")
run_grid_search(X_train_bin, y_train_bin, X_val_bin, y_val_bin,
↪   LogisticRegression(max_iter=1000), logreg_grid, "LogReg - 2 classes (binary without
↪   neutral or irrelevant)")
```

To operationalize our modeling approach, we implement a pipeline that combines TF-IDF vectorization with either a Support Vector Machine or a Logistic Regression classifier. We also design a grid search procedure to

optimize the hyperparameters of each model for each classification scenario.

Dataset Preparation for Each Scenario

We prepare separate training and validation sets for each of the three classification scenarios described earlier. The following code defines how data is filtered accordingly.

Pipeline and Grid Search Setup

We define a reusable pipeline that includes a `TfidfVectorizer` and a classifier, and perform grid search using cross-validation to find the best hyperparameters. Separate grids are defined for SVM and logistic regression, allowing for flexible experimentation.

Execution and Evaluation

We run the pipeline for both classifiers and across all three classification scenarios. Each run reports the best parameters found through cross-validation and displays the classification report on the validation set.

Results and Performance Analysis

In this section, we report the results of the classification experiments across the three defined scenarios (binary, ternary, and four-class) using both **Support Vector Machines (SVM)** and **Logistic Regression (LogReg)**. We focus on key metrics such as accuracy, precision, recall, and F1-score on the validation set.

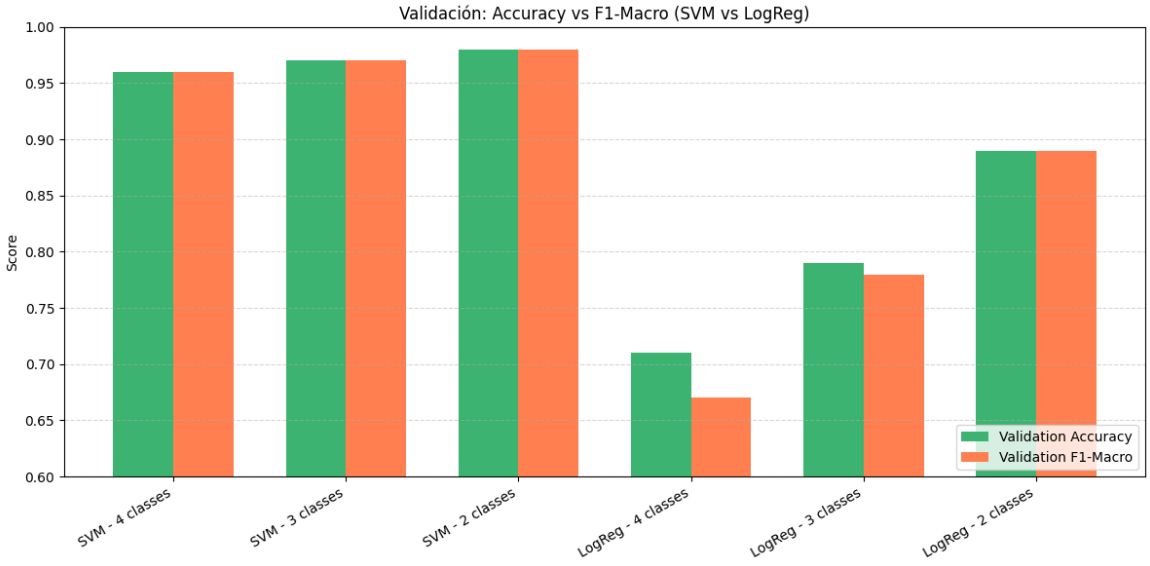

Figure 4

SVM Performance

- **SVM – 4-Class (with neutral and irrelevant)**
  Best Parameters: `C=0.1, ngram_range=(1,2), max_df=0.9, min_df=1`
  Validation Accuracy: 0.96
  All four classes were predicted with high precision and recall, especially the "Irrelevant" class (F1 = 0.96), showing that SVM is highly capable of handling multiclass sentiment classification even in more complex settings.
- **SVM – 3-Class (excluding irrelevant)**
  Best Parameters: Same as above
  Validation Accuracy: 0.97

Removing the irrelevant class led to a slight improvement in overall accuracy. The model maintained a consistent F1-score across the three remaining classes.

- **SVM – Binary (excluding neutral and irrelevant)**
  Best Parameters: Same as above
  Validation Accuracy: 0.98
  As expected, the binary setting yielded the highest accuracy and F1-score. The model distinguishes well between clear positive and negative sentiments.

Logistic Regression Performance

- **LogReg – 4-Class (with neutral and irrelevant)**
  Best Parameters: `C=0.1, solver='liblinear', ngram_range=(1,2), max_df=0.9, min_df=3`
  Validation Accuracy: 0.71
  Logistic Regression struggled with the four-class setup, particularly with the "Irrelevant" class (F1 = 0.46). Although it performed reasonably on the other classes, its overall macro-average was significantly lower than that of SVM.
- **LogReg – 3-Class (excluding irrelevant)**
  Best Parameters: Same as above
  Validation Accuracy: 0.79
  Performance improved when the irrelevant class was removed. The model showed the best balance between precision and recall on the neutral and positive classes.
- **LogReg – Binary (excluding neutral and irrelevant)**
  Best Parameters: `C=0.1, solver='liblinear', ngram_range=(1,2), max_df=0.9, min_df=1`
  Validation Accuracy: 0.89
  As with SVM, the binary setup resulted in the highest accuracy. However, LogReg remained slightly behind in performance when compared to SVM in this simpler classification task.

Comparison Summary

- **SVM consistently outperformed Logistic Regression** across all scenarios, particularly in the multiclass settings.
- The **binary classification task** resulted in the best performance for both models, as expected due to reduced complexity.
- Logistic Regression struggled with the **Irrelevant** class, indicating that this classifier may not generalize well in noisy environments.

These results suggest that SVM is a more robust choice for sentiment analysis in real-world settings, especially when the dataset includes ambiguous or off-topic content.

References

[1] Raschka, S., Liu, Y. & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*. Packt.